BÁO CÁO THỰC HÀNH KHAI THÁC DỮ LIỆU

BÀI TẬP LÝ THUYẾT TUẦN 1

Phạm Ngọc Hào - 23110146

Ngày 27 tháng 3 năm 2026

1 Nội dung thực hiện

Trong phần báo cáo này, ta sử dụng ba hàm preprocess_data1(), preprocess_data2() và preprocess_data3() để thực hiện xử lý dữ liệu lần lượt cho các phần tiếp theo. Trước hết, ta cần các sử dụng các thư viện cần thiết như sau:

```
import pandas as pd
import numpy as np
from sklearn import preprocessing
from sklearn.preprocessing import LabelEncoder, KBinsDiscretizer
from sklearn.decomposition import PCA
```

1.1 Bài 1: Tiền xử lý dữ liệu cơ bản

Trong bài này, dữ liệu nhân viên (data.csv) được xử lý qua các bước sau:

1. **Load dữ liệu**: Ta sử dụng hàm load_csv()dưới đây để nhập dữ liệu "data.csv".

```
def load_csv(path = "./data/data.csv"):
    try:
        df = pd.read_csv(path, delimiter = ",")
        df.set_index("ID", inplace = True)
        print("Loaded data.csv successfully\n")
        return df
    except Exception as e:
        print(f"Error: {e}")
        return None
```

Các phần xử lý dữ liệu tiếp theo sẽ được thực hiện trên hàm preprocess_data1(), ta sẽ load và xuất dữ liệu ra màn hình như sau bằng khối lệnh sau:

```
print("# Part 1:")
print("## 1. Load the csv file")
df = load_csv()
if df is None:
    print("Not found the file")
    return
print(df)
print()
```

Sau khi thực thi đoạn mã trên ta có kết quả như [1].

2. **Xử lý giá trị thiếu:** Sử dụng `fillna()` với giá trị trung bình (*mean*) cho cột "Age"và "Salary":

```
print("## 2. Preprocessing the missing data")
print(df.isnull().sum())
print(f"Total missing values: {df.isnull().sum().sum()}")
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
print("After precessing missing data")
print(df)
print()
```

Ta được kết quả là ID tại 3 cột Age là 32.1818 và tương tự với ID tại 7 và cột Salary với giá trị là 62272.727273 (xem hình [2]).

3. **Loại bỏ trùng lặp:** Sử dụng `drop_duplicates()` để loại bỏ các bản ghi giống hệt nhau:

```
print("## 3. Precessing the duplicated data")
print(f"Total duplicated values: {df.duplicated().sum()}")
df = df.drop_duplicates()
print("After eliminating the same line in the data frame")
print(df)
print()
```

Kết quả được trình bày qua hình [3]

4. **Mã hóa Categorical:** Sử dụng `get_dummies` để biến đổi các biến định danh thành các cột nhị phân (xem hình [4]):

```
print("## 4. Encoding categorical data with get_dummies")
df = pd.get_dummies(
        df,
        columns = ["Gender", "Department"],
        dtype = "int"
    )
print("After using get_dummies()\n", df)
print()
```

5. **Xử lý ngoại lai:** Áp dụng `RobustScaler` để giảm ảnh hưởng của các giá trị nhiễu (xem hình [5])

```
print("## 6. Processing outliers")
df1 = df.drop(["First Name", "Last Name", "day_of_week"], axis
    =1)
array = df1.values
# print(array)
print()
scaler = preprocessing.RobustScaler()
robust_df1 = scaler.fit_transform(array)
robust_df = pd.DataFrame(robust_df1)
print("Removing outliers\n", robust_df)
print()
```

6. **Chuẩn hóa:** Thực hiện Z-score normalization và Min-Max scaling (Kết quả xem hình [6])

```
print("## 7. Normalizing and Scaling data")
print("Normalizing data using z-score method (Standard)")

scaler = preprocessing.StandardScaler()
standard = scaler.fit_transform(array)
standard_df = pd.DataFrame(standard, index = df.index)
print("Normalizing data:\n", standard_df)
print()

print("Scaling data using minmax method")
scaler = preprocessing.MinMaxScaler()
minmax = scaler.fit_transform(array)
minmax_df = pd.DataFrame(minmax, index = df.index)
print("Scaling data:\n", minmax_df)
print()
```

7. **Rời rạc hóa:** Chia cột dữ liệu bằng `pd.cut()` (Equi-width) và `pd.qcut()` (Equi-depth) (Xem hình [7]):

```
print("## 8. Discretizing data")
std_df = pd.DataFrame(standard_df.copy(), index = df.index)
print("### 1. 10 equi-width ranges with first column of
    standard_df")
df2 = std_df.copy()
df2["equi-width_column0"] = pd.cut(x = df2.iloc[:, 0], bins = 10)
print("Digitzing column 0 with 10 equi-with ranges:\n", df2)
print()
print("### 2. 10 equi-depth ranges with first column of
    standard_df")
df3 = std_df.copy()
df3["equi-depth_column0"] = pd.qcut(df3.iloc[:, 0], q = 10)
print("Discretizing column 0 using 10 equi-depth range:\n", df3)
print("="*91)
print("\n\n")
```

1.2 Bài 2: Thực hành trên tập dữ liệu Arrhythmia

Trong bài này, ta thực hiện tiền xử lý trên tập dữ liệu (*Arrhythmia*) từ UCI. Quy trình được thực hiện thông qua hàm `preprocess_data2()` với các bước cụ thể như sau:

1. **Load dữ liệu**: Dữ liệu được tải từ tệp `arrhythmia.data`. Do dữ liệu này sử dụng ký tự "?"để đánh dấu các giá trị thiếu, ta cần thiết lập tham số `na_values` khi đọc file.

```
def load_data(path = "./data/arrhythmia.data"):
    try:
        df = pd.read_csv(path, delimiter= ",", header = None,
    na_values = ['?'])
        print("Loaded arrhythmia.data successfully\n")
        return df
    except Exception as e:
        print(f"Error: {e}")
        return None
```

Kết quả như hình [8].

2. **Xử lý giá trị thiếu**: Ta tiến hành kiểm tra tổng số lượng giá trị *NaN* và thực hiện điền đầy bằng phương pháp tính trung bình (*mean*) trên toàn bộ DataFrame (xem hình [9]).

```
print("## 2. Preprocessing the missing data")
print(f"Total the NA values: {df.isna().sum().sum()}")
# Dien day cac gia tri thieu bang trung binh cua cot
df = df.fillna(df.mean())
print(f"Total the NA values after processing: {df.isna().sum().
    sum()}\n")
```

3. **Xử lý ngoại lai và Chuẩn hóa**: Trước tiên, ta loại bỏ cột nhãn cuối cùng (cột phân loại bệnh). Sau đó, sử dụng `RobustScaler` để xử lý các điểm dữ liệu dị biệt và `StandardScaler` để chuẩn hóa dữ liệu về phân phối chuẩn (xem kết quả tại hình [10] và [11]).

```
print("## 4. Processing the outliers")
X = df.drop(df.columns[-1], axis = 1)
scaler = preprocessing.RobustScaler()
robust_arr = scaler.fit_transform(X)

print("## 5. Normalizing and Scaling data")
scaler_std = preprocessing.StandardScaler()
standard = scaler_std.fit_transform(robust_arr)
standard_df = pd.DataFrame(standard, index = df.index)
```

4. **Rời rạc hóa đa biến**: Sử dụng lớp `KBinsDiscretizer` với chiến lược 'quantile' (Equi-depth) để chia các đặc trưng số thành 10 khoảng có số lượng mẫu bằng nhau (xem hình [12]).

```
print("## 6. Discretizing data")
# Chon cac cot co su bien thien (nunique > 1)
selector = standard_df.nunique() > 1
train_df = standard_df.loc[:, selector]
est = KBinsDiscretizer(n_bins=10, encode="ordinal", strategy="
    quantile")
data_discrete = est.fit_transform(train_df)
print(data_discrete)
```

1.3 Bài 3: Thực hành trên tập dữ liệu Musk

Bài thực hành này áp dụng kỹ thuật PCA trên tập dữ liệu *Musk* để giảm số lượng đặc trưng nhưng vẫn giữ lại tối đa lượng thông tin (phương sai) của dữ liệu gốc. Quy trình thực hiện trong hàm `preprocess_data3()`:

1. **Tiền xử lý dữ liệu Musk**: Ta loại bỏ các cột định danh (ID) và cột nhãn mục tiêu để tập trung vào các đặc trưng hóa học của phân tử.

```
# Loai bo cac cot khong phai dac trung (0, 1) va cot nhan (168)
X = df.drop([0, 1, 168], axis = 1)
X = X.fillna(X.mean())
X = X.drop_duplicates()
```

2. **Thực hiện thuật toán PCA**: Sau khi chuẩn hóa, ta sử dụng thư viện *sklearn* để thực hiện phép xoay trục tọa độ, tìm ra các vector riêng (*Eigenvectors*) và trị riêng (*Eigenvalues*) tương ứng.

```python
print("## 7. Applying the PCA Algorithm")
pca = PCA()
pca.fit(standard)

print("Result:")
print(f"Eigenvalues:\n{pca.explained_variance_}\n")
print(f"Eigenvectors:\n{pca.components_}")
```

Các trị riêng thu được cho biết mức độ đóng góp của từng thành phần chính vào việc giải thích sự biến thiên của dữ liệu. Kết quả chi tiết được trình bày qua hình [13] và [14].

2 Kết quả thực thi

2.1 Kết quả thực thi khi chạy chương trình của preprocess_data1()

Hình 1: Load dữ liệu

```
phnghao@NgocHao:~/projects/data_mining$  source /home/phnghao/projects/data_mining/DMenv/bin/activate
(DMenv) phnghao@NgocHao:~/projects/data_mining$ cd Week01
(DMenv) phnghao@NgocHao:~/projects/data_mining/Week01$ python3 week01.py
# Part 1:
## 1. Load the csv file
Loaded data.csv successfully

   First Name Last Name   Age Gender Department   Salary Date of Joining
ID
1        John       Doe  25.0      M      Sales  50000.0      01/01/2020
2        Jane     Smith  30.0      F  Marketing  60000.0      06/01/2018
3         Bod   Johnson  45.0      M         HR  70000.0      09/01/2016
4       Alice  Williams  33.0      F         IT  80000.0      02/01/2017
5       James     Brown  27.0      M      Sales  55000.0      03/01/2019
6       Sarah       Lee   NaN      F  Marketing  65000.0      12/01/2018
7     Michael     Davis  39.0      M         HR      NaN      08/01/2015
8       Susan    Miller  42.0      F         IT  90000.0      11/01/2014
9       David    Wilson  28.0      M      Sales  60000.0      05/01/2020
10      Emily     Brown  35.0      F  Marketing  55000.0      04/01/2017
11       John       Doe  25.0      M      Sales  50000.0      01/01/2020
12       John       Doe  25.0      M      Sales  50000.0      01/01/2020
```

Hình 2: Kết quả khi xử lí giá trị missing

```
## 2. Preprocessing the missing data
First Name         0
Last Name          0
Age                1
Gender             0
Department         0
Salary             1
Date of Joining    0
dtype: int64
Total missing values: 2
After precessing missing data
   First Name Last Name        Age Gender Department        Salary Date of Joining
ID
1        John       Doe  25.000000      M      Sales  50000.000000      01/01/2020
2        Jane     Smith  30.000000      F  Marketing  60000.000000      06/01/2018
3         Bod   Johnson  45.000000      M         HR  70000.000000      09/01/2016
4       Alice  Williams  33.000000      F         IT  80000.000000      02/01/2017
5       James     Brown  27.000000      M      Sales  55000.000000      03/01/2019
6       Sarah       Lee  32.181818      F  Marketing  65000.000000      12/01/2018
7     Michael     Davis  39.000000      M         HR  62272.727273      08/01/2015
8       Susan    Miller  42.000000      F         IT  90000.000000      11/01/2014
9       David    Wilson  28.000000      M      Sales  60000.000000      05/01/2020
10      Emily     Brown  35.000000      F  Marketing  55000.000000      04/01/2017
11       John       Doe  25.000000      M      Sales  50000.000000      01/01/2020
12       John       Doe  25.000000      M      Sales  50000.000000      01/01/2020
```

Hình 3: Kết quả khi xử lí giá trị duplicate

```
## 3. Precessing the duplicated data
Total duplicated values: 2
After eliminating the same line in the data frame
   First Name Last Name        Age Gender Department        Salary Date of Joining
ID
1        John       Doe  25.000000      M      Sales  50000.000000      01/01/2020
2        Jane     Smith  30.000000      F  Marketing  60000.000000      06/01/2018
3         Bod   Johnson  45.000000      M         HR  70000.000000      09/01/2016
4       Alice  Williams  33.000000      F         IT  80000.000000      02/01/2017
5       James     Brown  27.000000      M      Sales  55000.000000      03/01/2019
6       Sarah       Lee  32.181818      F  Marketing  65000.000000      12/01/2018
7     Michael     Davis  39.000000      M         HR  62272.727273      08/01/2015
8       Susan    Miller  42.000000      F         IT  90000.000000      11/01/2014
9       David    Wilson  28.000000      M      Sales  60000.000000      05/01/2020
10      Emily     Brown  35.000000      F  Marketing  55000.000000      04/01/2017
```

Hình 4: Kết quả khi mã hóa Categorical

```
## 4. Encoding categorical data with get_dummies
After using get_dummies()
   First Name Last Name        Age        Salary Date of Joining  Gender_F  Gender_M  Department_HR  Department_IT  Department_Marketing  Department_Sales
ID
1        John       Doe  25.000000  50000.000000      01/01/2020         0         1              0              0                     0                 1
2        Jane     Smith  30.000000  60000.000000      06/01/2018         1         0              0              0                     1                 0
3         Bod   Johnson  45.000000  70000.000000      09/01/2016         0         1              1              0                     0                 0
4       Alice  Williams  33.000000  80000.000000      02/01/2017         1         0              0              1                     0                 0
5       James     Brown  27.000000  55000.000000      03/01/2019         0         1              0              0                     0                 1
6       Sarah       Lee  32.181818  65000.000000      12/01/2018         1         0              0              0                     1                 0
7     Michael     Davis  39.000000  62272.727273      08/01/2015         0         1              1              0                     0                 0
8       Susan    Miller  42.000000  90000.000000      11/01/2014         1         0              0              1                     0                 0
9       David    Wilson  28.000000  60000.000000      05/01/2020         0         1              0              0                     0                 1
10      Emily     Brown  35.000000  55000.000000      04/01/2017         1         0              0              0                     1                 0
```

Hình 5: Kết quả sau khi xử lí giá trị ngoại lai

```
Removing outliers
          0         1    2    3    4    5         6         7         8
0 -0.799043 -0.890909 -0.5  0.5  0.0  0.0  0.000000  1.333333 -0.818182
1 -0.272727 -0.090909  0.5 -0.5  0.0  0.0  1.333333  0.000000  0.090909
2  1.306220  0.709091 -0.5  0.5  1.0  0.0  0.000000  0.000000  0.636364
3  0.043062  1.509091  0.5 -0.5  0.0  1.0  0.000000  0.000000 -0.636364
4 -0.588517 -0.490909 -0.5  0.5  0.0  0.0  0.000000  1.333333 -0.454545
5 -0.043062  0.309091  0.5 -0.5  0.0  0.0  1.333333  0.000000  1.181818
6  0.674641  0.090909 -0.5  0.5  1.0  0.0  0.000000  0.000000  0.454545
7  0.990431  2.309091  0.5 -0.5  0.0  1.0  0.000000  0.000000  1.000000
8 -0.483254 -0.090909 -0.5  0.5  0.0  0.0  0.000000  1.333333 -0.090909
9  0.253589 -0.490909  0.5 -0.5  0.0  0.0  1.333333  0.000000 -0.272727
```

Hình 6: Kết quả sau khi chuẩn hóa bằng z-score và min-max

```
## 7. Normalizing and Scaling data
Normalizing data using z-score method (Standard)
Normalizing data:
           0         1    2    3    4    5         6         7         8
ID
1  -1.369782 -1.264391 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -1.420508
2  -0.575077 -0.405854  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654 -0.027853
3   1.809037  0.452683 -1.0  1.0  2.0 -0.5 -0.654654 -0.654654  0.807740
4  -0.098254  1.311220  1.0 -1.0 -0.5  2.0 -0.654654 -0.654654 -1.141977
5  -1.051900 -0.835122 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -0.863446
6  -0.228297  0.023415  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654  1.643333
7   0.855391 -0.210732 -1.0  1.0  2.0 -0.5 -0.654654 -0.654654  0.529209
8   1.332214  2.169757  1.0 -1.0 -0.5  2.0 -0.654654 -0.654654  1.364802
9  -0.892959 -0.405854 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -0.306384
10  0.219627 -0.835122  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654 -0.584915

Scaling data using minmax method
Scaling data:
           0         1    2    3    4    5    6    7         8
ID
1   0.000000  0.000000  0.0  1.0  0.0  0.0  0.0  1.0  0.000000
2   0.250000  0.250000  1.0  0.0  0.0  0.0  1.0  0.0  0.454545
3   1.000000  0.500000  0.0  1.0  1.0  0.0  0.0  0.0  0.727273
4   0.400000  0.750000  1.0  0.0  0.0  1.0  0.0  0.0  0.090909
5   0.100000  0.125000  0.0  1.0  0.0  0.0  0.0  1.0  0.181818
6   0.359091  0.375000  1.0  0.0  0.0  0.0  1.0  0.0  1.000000
7   0.700000  0.306818  0.0  1.0  1.0  0.0  0.0  0.0  0.636364
8   0.850000  1.000000  1.0  0.0  0.0  1.0  0.0  0.0  0.909091
9   0.150000  0.250000  0.0  1.0  0.0  0.0  0.0  1.0  0.363636
10  0.500000  0.125000  1.0  0.0  0.0  0.0  1.0  0.0  0.272727
```

Hình 7: Kết quả sau khi rời rạc hóa tập dữ liệu bằng Equi-width và Equi-depth

```
## 8. Discretizing data
### 1. 10 equi-width ranges with first column of standard_df
Digitzing column 0 with 10 equi-with ranges:
            0         1    2    3    4    5         6         7         8 equi-width_column0
ID
1  -1.369782 -1.264391 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -1.420508   (-1.373, -1.052]
2  -0.575077 -0.405854  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654 -0.027853   (-0.734, -0.416]
3   1.809037  0.452683 -1.0  1.0  2.0 -0.5 -0.654654 -0.654654  0.807740     (1.491, 1.809]
4  -0.098254  1.311220  1.0 -1.0 -0.5  2.0 -0.654654 -0.654654 -1.141977    (-0.0983, 0.22]
5  -1.051900 -0.835122 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -0.863446   (-1.373, -1.052]
6  -0.228297  0.023415  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654  1.643333  (-0.416, -0.0983]
7   0.855391 -0.210732 -1.0  1.0  2.0 -0.5 -0.654654 -0.654654  0.529209     (0.538, 0.855]
8   1.332214  2.169757  1.0 -1.0 -0.5  2.0 -0.654654 -0.654654  1.364802     (1.173, 1.491]
9  -0.892959 -0.405854 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -0.306384   (-1.052, -0.734]
10  0.219627 -0.835122  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654 -0.584915    (-0.0983, 0.22]

### 2. 10 equi-depth ranges with first column of standard_df
Discretizing column 0 using 10 equi-depth range:
            0         1    2    3    4    5         6         7         8 equi-depth_column0
ID
1  -1.369782 -1.264391 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -1.420508   (-1.371, -1.084]
2  -0.575077 -0.405854  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654 -0.027853    (-0.67, -0.367]
3   1.809037  0.452683 -1.0  1.0  2.0 -0.5 -0.654654 -0.654654  0.807740      (1.38, 1.809]
4  -0.098254  1.311220  1.0 -1.0 -0.5  2.0 -0.654654 -0.654654 -1.141977   (-0.163, 0.0289]
5  -1.051900 -0.835122 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -0.863446   (-1.084, -0.925]
6  -0.228297  0.023415  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654  1.643333   (-0.367, -0.163]
7   0.855391 -0.210732 -1.0  1.0  2.0 -0.5 -0.654654 -0.654654  0.529209      (0.41, 0.951]
8   1.332214  2.169757  1.0 -1.0 -0.5  2.0 -0.654654 -0.654654  1.364802      (0.951, 1.38]
9  -0.892959 -0.405854 -1.0  1.0 -0.5 -0.5 -0.654654  1.527525 -0.306384    (-0.925, -0.67]
10  0.219627 -0.835122  1.0 -1.0 -0.5 -0.5  1.527525 -0.654654 -0.584915     (0.0289, 0.41]
==========================================================================================
```

2.2 Kết quả thực thi khi chạy chương trình của preprocess_data2()

Hình 8: Load dữ liệu Arrhythmia

```
# Part 2:

## 1. Load the data
Loaded arrhythmia.data successfully

   0   1   2   3   4   5   6   7   8   9    10   11   12   13   14  15  16  17  18  ...   261  262  263  264  265  266   267   268  269  270   271  272  273  274  275  276   277   278
279
0  75   0 190  80  91 193 371 174 121 -16  13.0 64.0 -2.0  NaN 63.0   0  52  44   0 ...  13.5 -4.0  0.0  0.0  0.9  3.9  25.5  62.9 -0.3  0.0   9.0 -0.9  0.0  0.0  0.9  2.9  23.3  49.4
  8
1  56   1 165  64  81 174 401 149  39  25  37.0 -17.0 31.0  NaN 53.0   0  48   0   0 ...  11.0 -2.4  0.0  0.0  0.4  2.6  21.6  43.4 -0.5  0.0   8.5  0.0  0.0  0.0  0.2  2.1  20.4  38.8
  6
2  54   0 172  95 138 163 386 185 102  96  34.0  70.0 66.0 23.0 75.0   0  40  80   0 ...  11.1 -3.4  0.0  0.0  0.4  3.4  11.5  48.2  0.9  0.0   9.5 -2.4  0.0  0.0  0.3  3.4  12.3  49.0
 10
3  55   0 175  94 100 202 380 179 143  28  11.0  -5.0 20.0  NaN 71.0   0  72  20   0 ...  15.2 -3.7  0.0  0.0  0.6  3.0  36.8  68.0  0.1  0.0  12.2 -2.2  0.0  0.0  0.4  2.6  34.6  61.6
  1
4  75   0 190  80  88 181 360 177 103 -16  13.0  61.0  3.0  NaN  NaN   0  48  40   0 ...   9.1 -0.9  0.0  0.0 -0.2  2.9  21.7  48.9 -0.4  0.0  13.1 -3.6  0.0  0.0 -0.1  3.9  25.4  62.8
  7
```

Hình 9: Xử lí giá trị missing

```
## 2. Preprocessing the missing data
Check the NA values before processing the NA data
0      0
1      0
2      0
3      0
4      0
      ..
275    0
276    0
277    0
278    0
279    0
Length: 280, dtype: int64
Total the NA values: 408
Check the NA values after processing the NA data
0      0
1      0
2      0
3      0
4      0
      ..
275    0
276    0
277    0
278    0
279    0
Length: 280, dtype: int64
Total the NA values: 0
```

Hình 10: Xử lí giá trị ngoại lai

```
## 4. Processing the outliers
Data after processing outliers by RobustScaler
       0    1    2     3        4        5        6        7        8        9        10       11  ...      267      268  269  270      271      272  273  274      275     276
    277      278
0  1.272727 -1.0  2.6  0.60  0.357143  1.090909  0.117647  0.387097  1.304348 -0.899598 -0.571429  0.413043  ...  0.371912  0.994652 -0.2  0.0  0.043478  0.095238  0.0  0.0  1.333333  0.96875  0.
361739  0.911983
1  0.409091  0.0  0.1 -0.20 -0.357143  0.515152  1.000000 -0.419355 -2.260870 -0.240964 -0.081633 -3.108696  ...  0.169051  0.398778 -0.6  0.0 -0.065217  0.523810  0.0  0.0 -1.000000  0.46875  0.
160000  0.462354
2  0.318182 -1.0  0.8  1.35  3.714286  0.181818  0.558824  0.741935  0.478261  0.899598 -0.142857  0.673913  ... -0.356307  0.545455  2.2  0.0  0.152174 -0.619048  0.0  0.0 -0.666667  1.28125 -0.
403478  0.895016
3  0.363636 -1.0  1.1  1.30  1.000000  1.363636  0.382353  0.548387  2.260870 -0.192771 -0.612245 -2.586957  ...  0.959688  1.150497  0.6  0.0  0.739130 -0.523810  0.0  0.0 -0.333333  0.78125  1.
147826  1.429480
4  1.272727 -1.0  2.6  0.60  0.142857  0.727273 -0.205882  0.483871  0.521739 -0.899598 -0.571429  0.282609  ...  0.174252  0.566845 -0.4  0.0  0.934783 -1.190476  0.0  0.0 -2.000000  1.59375  0.
507826  1.480382
```

Hình 11: Chuẩn hóa dữ liệu

```
## 5. Normalizing and Scaling data
Normalizing data using z-score method (Standard)
Normalizing data:
        0         1         2         3         4         5         6         7         8         9   ...       269       270       271       272       273  274       275       276       277
    278
0  1.734439 -1.107520  0.641327  0.713814  0.135505  0.844945  0.113709  0.113809  1.201469 -1.094661  ...  0.004037  0.508843 -0.013839  0.278621 -0.079546  0.0  1.109553  1.177737  0.294603  1.
078670
1  0.579312  0.902918 -0.031998 -0.251644 -0.516072  0.420769  1.013301 -0.588564 -1.977064 -0.191203  ... -0.327703  0.508843 -0.157972  0.728573 -0.079546  0.0 -0.906889  0.616126  0.079613  0.
504874
2  0.457720 -1.107520  0.156533  1.618932  3.197915  0.175193  0.563505  0.422853  0.464980  1.373324  ...  1.994474  0.508843  0.130294 -0.471299 -0.079546  0.0 -0.618826  1.528744 -0.520878  1.
057018
3  0.518516 -1.107520  0.237332  1.558590  0.721924  1.045871  0.383587  0.254284  2.054247 -0.125096  ...  0.667516  0.508843  0.908612 -0.371310 -0.079546  0.0 -0.330763  0.967133  1.132324  1.
739077
4  1.734439 -1.107520  0.641327  0.713814 -0.059968  0.577044 -0.216141  0.198094  0.503742 -1.094661  ... -0.161833  0.508843  1.168051 -1.071235 -0.079546  0.0 -1.771079  1.879751  0.450286  1.
804035

[5 rows x 279 columns]

Scaling data using minmax method
Scaling data:
        0   1         2         3         4         5         6         7         8         9        10  ...       268       269  270       271       272  273  274      275       276       27
7       278
0  0.903614  0.0  0.125926  0.435294  0.270677  0.368321  0.501805  0.241758  0.590244  0.457478  0.533708  ...  0.660574  0.638554  1.0  0.381356  0.968531  0.0  0.0  0.53125  0.741667  0.507519
  0.569579
1  0.674699  1.0  0.088889  0.341176  0.195489  0.332061  0.610108  0.150183  0.190244  0.577713  0.601124  ...  0.558747  0.614458  1.0  0.360169  1.000000  0.0  0.0  0.31250  0.675000  0.485714
  0.500971
2  0.650602  0.0  0.099259  0.523529  0.624060  0.311069  0.555957  0.282051  0.497561  0.785924  0.592697  ...  0.583812  0.783133  1.0  0.402542  0.916084  0.0  0.0  0.34375  0.783333  0.424812
  0.566990
3  0.662651  0.0  0.103704  0.517647  0.338346  0.385496  0.534296  0.260073  0.697561  0.586510  0.528090  ...  0.687206  0.686747  1.0  0.516949  0.923077  0.0  0.0  0.37500  0.716667  0.592481
  0.648544
4  0.903614  0.0  0.125926  0.435294  0.248120  0.345420  0.462094  0.252747  0.502439  0.457478  0.533708  ...  0.587467  0.626506  1.0  0.555085  0.874126  0.0  0.0  0.21875  0.825000  0.523308
  0.656311
```

Hình 12: Chuẩn hóa dữ liệu

```
## 6. Discretizing data
[[9. 0. 8. ... 9. 6. 8.]
 [7. 0. 5. ... 7. 5. 7.]
 [6. 0. 7. ... 9. 2. 8.]
 ...
 [2. 0. 5. ... 3. 0. 0.]
 [1. 0. 1. ... 8. 7. 8.]
 [9. 0. 3. ... 6. 5. 6.]]
==================================
```

2.3 Kết quả thực thi khi chạy chương trình của preprocess_data3()

Hình 13: Eigen values

```
## 7. Applying the PCA Algorithm
Result:
Eigenvalues:
[4.86609639e+01 2.95142984e+01 8.17064063e+00 7.22600871e+00
 5.72416553e+00 5.25232015e+00 4.85741902e+00 4.46546662e+00
 4.02738194e+00 3.87390950e+00 3.12843963e+00 2.97469269e+00
 2.58203120e+00 2.41322457e+00 2.13551413e+00 2.09564398e+00
 1.85145826e+00 1.73429116e+00 1.49293357e+00 1.41899692e+00
 1.17417158e+00 1.08511661e+00 1.06597317e+00 1.05804572e+00
 1.00750244e+00 9.78261790e-01 8.62748276e-01 8.49408890e-01
 7.49694969e-01 7.34319126e-01 6.81020685e-01 6.04487806e-01
 5.98641242e-01 5.89950082e-01 5.55669959e-01 5.05748861e-01
 4.72743715e-01 4.38355299e-01 4.05767917e-01 3.97767266e-01
 3.84588785e-01 3.69091256e-01 3.20937783e-01 3.01263974e-01
 2.83022041e-01 2.76335301e-01 2.61403336e-01 2.53088402e-01
 2.46739335e-01 2.34356466e-01 2.14717349e-01 1.94748090e-01
 1.85788838e-01 1.82432577e-01 1.70900821e-01 1.61693606e-01
 1.53007234e-01 1.50910854e-01 1.43424953e-01 1.33156418e-01
 1.29163619e-01 1.17871412e-01 1.14540840e-01 1.12003961e-01
 1.08433177e-01 1.06768235e-01 1.02035682e-01 9.93253172e-02
 9.56701076e-02 9.12541497e-02 8.56468432e-02 7.95660745e-02
 7.49201466e-02 7.01636395e-02 6.92033947e-02 6.84584411e-02
 6.43451505e-02 6.13682768e-02 5.88559503e-02 5.79180741e-02
 5.41334841e-02 5.39124576e-02 4.86268379e-02 4.81102280e-02
 4.53048262e-02 4.16624519e-02 4.07915658e-02 3.83036109e-02
 3.60623955e-02 3.37434586e-02 3.18027435e-02 3.10347617e-02
 2.97617308e-02 2.93594254e-02 2.85393431e-02 2.72320963e-02
 2.61974189e-02 2.40390265e-02 2.37748703e-02 2.24776986e-02
 2.21676612e-02 2.17550700e-02 2.13876598e-02 1.98382507e-02
 1.90342750e-02 1.84803208e-02 1.71277898e-02 1.66557231e-02
 1.63664952e-02 1.58047924e-02 1.50547194e-02 1.44245249e-02
 1.38192588e-02 1.34738742e-02 1.30477549e-02 1.27276882e-02
 1.22826286e-02 1.18466285e-02 1.11579722e-02 1.05308837e-02
 1.01083507e-02 9.84812490e-03 9.70059518e-03 8.68461449e-03
 8.29628591e-03 8.12328912e-03 7.54061593e-03 7.27755823e-03
 7.12403897e-03 6.82091693e-03 6.66512046e-03 6.33594128e-03
 5.98513930e-03 5.73262381e-03 5.43470803e-03 5.20447017e-03
 5.08831727e-03 4.87410008e-03 4.77016269e-03 4.42606852e-03
 4.24887409e-03 3.93969385e-03 3.72566406e-03 3.49978392e-03
 3.42935972e-03 3.06902602e-03 3.00915402e-03 2.81647705e-03
 2.66160409e-03 2.42479341e-03 2.31764935e-03 2.06267220e-03
 1.79476963e-03 1.56183470e-03 1.42565256e-03 1.28190381e-03
 1.25632036e-03 1.17336627e-03 1.04082139e-03 9.36155424e-04
 8.26687179e-04 6.55163563e-04 6.30936363e-04 5.25489627e-04
 4.44560080e-04 2.58327739e-04]
```

Hình 14: Eigen vector

```
Eigenvectors:
[[-2.34916723e-03 -7.86437236e-02 -1.16342551e-01 ... -3.09133650e-04
  -1.33627782e-02  2.05503031e-03]
 [-1.05988365e-01  2.37660758e-02  4.49585663e-02 ... -6.23933984e-02
  -8.31164658e-02  2.22595257e-02]
 [-1.01296520e-02  1.30101726e-01  1.69779238e-02 ...  4.50653349e-02
   2.30852032e-02  1.78885318e-01]
 ...
 [ 5.41989194e-03  1.44846031e-02  1.35682200e-03 ... -1.08392703e-03
   3.04157994e-03  2.35485895e-05]
 [ 1.78056628e-05  1.30852123e-04  4.17367666e-02 ...  6.64131650e-04
  -1.32117750e-03 -1.43502498e-04]
 [-1.46577475e-04 -1.48682924e-03 -3.78461582e-02 ...  3.70446743e-04
  -9.62244625e-04  8.10713896e-04]]
==========================================================================
```

3 Kết luận

Qua buổi thực hành, ta có thể hiểu rõ tầm quan trọng của việc làm sạch dữ liệu trước khi đưa vào các mô hình. Việc chuẩn hóa và xử lý ngoại lai giúp mô hình ổn định hơn, trong khi PCA giúp tối ưu hóa không gian đặc trưng nhưng vẫn bảo toàn được lượng thông tin lớn nhất.